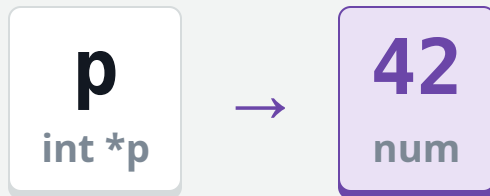




## CAPÍTULO 4

# Funções e ponteiros

Como dividir um programa em blocos reutilizáveis —  
e como uma função pode alterar diretamente as  
variáveis de quem a chamou.



# O que vamos ver

## 01 Funções

dividir o programa em blocos reutilizáveis, com parâmetros e retorno

---

## 02 Pilha de execução

o que acontece na memória a cada chamada de função

---

## 03 Ponteiros

uma variável que guarda o endereço de outra

---

## 04 Ponteiros como parâmetro

como uma função "retorna" mais de um valor

---

## 05 static e variáveis globais

memória que sobrevive entre chamadas

---

## 06 Recursão

uma função que chama a si mesma até um caso-base

---

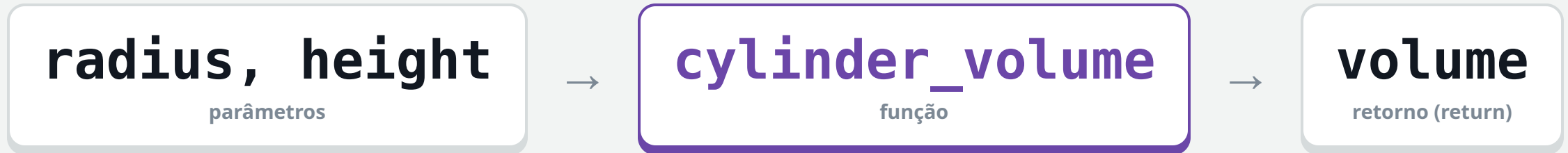
## 07 Pré-processador e macros

texto substituído antes mesmo da compilação

---

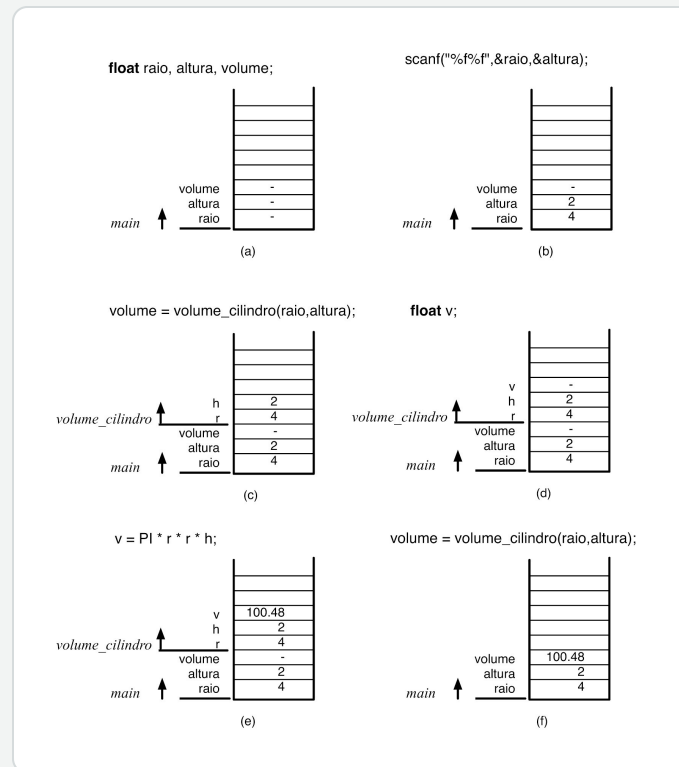
# Uma função é uma caixa

Parâmetros entram por cópia; o return devolve um único valor a quem chamou.



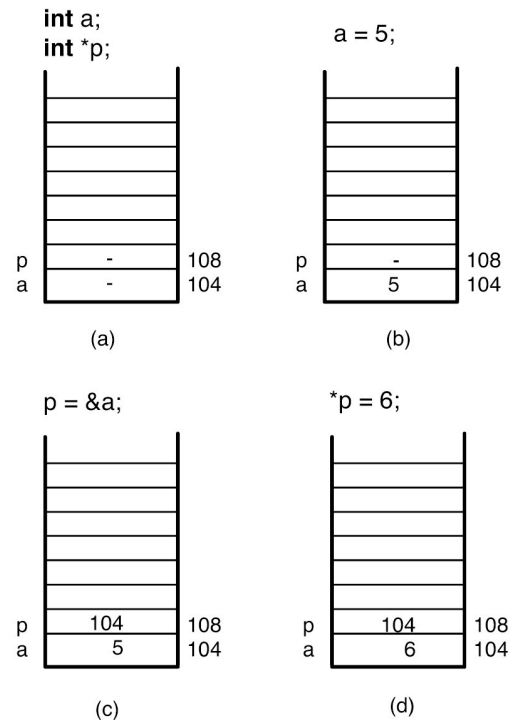
# Cada chamada empilha uma moldura

Parâmetros e variáveis locais vivem numa região de memória que desaparece quando a função retorna.



# Um ponteiro guarda um endereço

& obtém o endereço de uma variável; \* acessa — ou altera — o valor guardado nele.



# Por cópia, ou pelo endereço?

Uma função só altera o que recebeu por endereço — nunca uma cópia.

## **func(num)**

Recebe uma **cópia** de num — alterar essa cópia não muda o original.

```
num continua 10
```

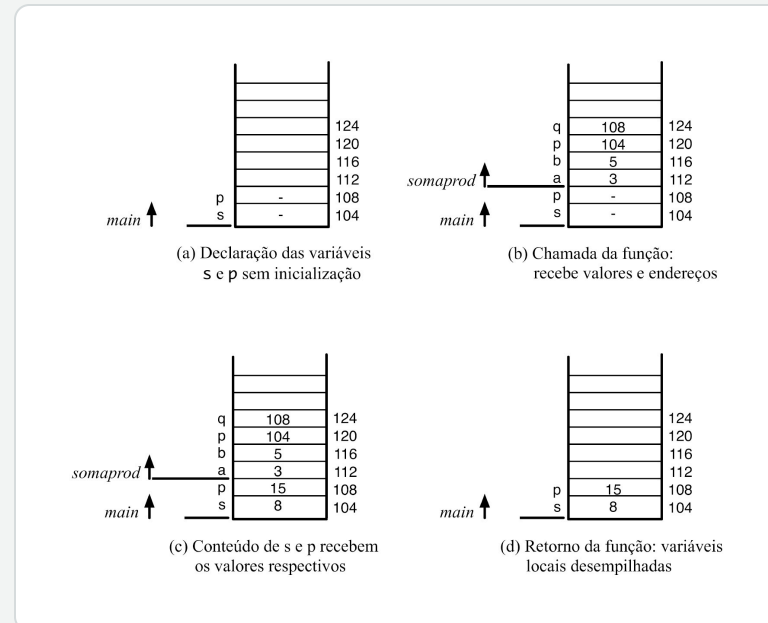
## **func(&num)**

Recebe o **endereço** de num — escreve através de \*ptr na variável original.

```
num passa a valer 100
```

# A porta dos fundos para retornar mais de um valor

Recebendo os endereços de `sum` e `difference`, a função escreve os dois resultados diretamente neles.



# Onde cada variável mora — e por quanto tempo

## local

Nasce e morre a cada chamada da função — some quando ela retorna.

```
vida: 1 chamada
```

## static (local)

Inicializada uma única vez; preserva o valor entre chamadas, mas continua visível só ali.

```
vida: todo o programa
```

## global

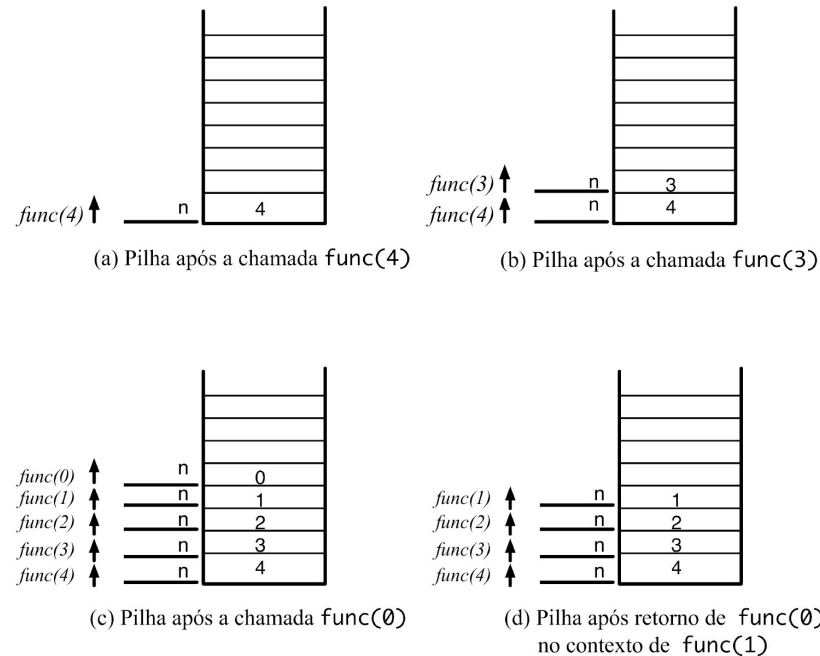
Existe do início ao fim do programa, visível — e alterável — por qualquer função.

```
vida: todo o programa
```



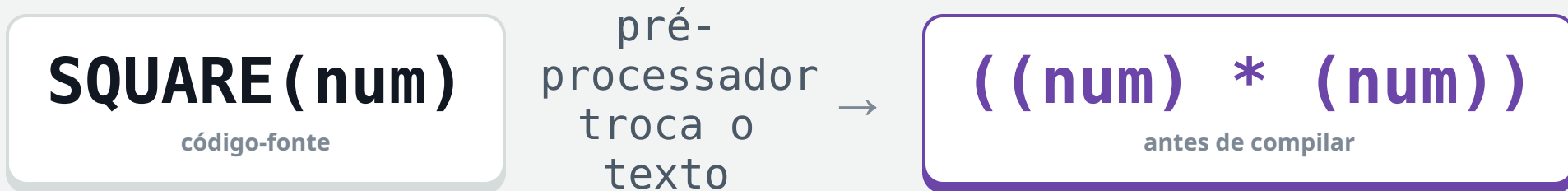
# Uma função que chama a si mesma

Cada chamada recursiva empilha sua própria moldura — até um caso-base interromper a cadeia.



# Texto substituído antes de compilar

#define não é uma função — não existe chamada, parâmetro ou retorno de verdade.



AGORA É COM VOCÊ

# Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada. Depois, teste o que aprendeu no quiz.



**Notas de Aula e Exemplos** [thiagopx.github.io/ed/02\\_functions\\_and\\_pointers](https://thiagopx.github.io/ed/02_functions_and_pointers)



O quiz de fixação está na mesma página.